NextWatch: Smart Watchlist & OTT Explorer

Project report: a full-stack watchlist app for movies and TV shows

Stack: React 18 + Vite + Material UI (client), Node.js + Express + MongoDB (server), TMDB API (data)

1. Overview

NextWatch lets a user sign up, browse and search movies and TV shows from The Movie Database (TMDB), and save them to a personal watchlist. Each saved title has a status (plan to watch, watching, completed, dropped), a rating out of 10, and a review. The app shows which streaming services carry a title in India, and it recommends titles based on the user's highest-rated items.

The project has two parts: a React single-page app in *client/*, and a REST API in *server/*. The API stores user accounts and watchlist items in MongoDB, and proxies TMDB requests so the TMDB key stays on the server.

2. Features

Area	What it does
Accounts	Sign up, log in and log out. Passwords are hashed with bcrypt. Sessions use a JWT that lasts one day.
Dashboard	Completion rate, count of pending titles, total saved, a hero card for the current or next title (with a 'Mark as completed' button), and the user's highest-rated titles.
Explore	Trending titles (day or week) and multi-type search for movies and TV, with paged loading.
Watchlist	Titles grouped into Currently Watching, The Queue (Plan to Watch) and Archive (Completed or Dropped). Status and rating can be changed in place. Plan-to-watch titles get a 'Streaming Now' badge when they are available on a streaming service in India.
For You	Recommendations from TMDB, seeded by up to three of the user's best-rated or completed titles. Falls back to weekly trending titles when the user has no rated items.
Details	A dialog per title with overview, genres, runtime or season count, rating, streaming providers in India, and a link to IMDB.
Theme	Light and dark modes, saved in the browser.

3. Architecture

The browser calls the API at *VITE_API_URL* (default *http://localhost:5000/api*). Each request carries the JWT in an Authorization header, added by an Axios interceptor. The API checks the token in *middleware/auth.js* before it runs any protected route. The explore routes call TMDB with the server-side key, so the key never reaches the browser.

Layer	Files	Role
Client entry	main.jsx, App.jsx	Providers (theme, auth), routes, protected pages
Client API	api/axios.js	Axios instance with base URL and token header
Client state	context/AuthContext.jsx	Current user; login, register and logout; checks the saved token on load
Client UI	components/*, pages/*	Navigation, cards, dialogs and the six pages
Server entry	server.js	Express app, CORS, JSON body parsing, routes, MongoDB connection
Auth	routes/auth.js, middleware/auth.js	Register, login, current user; token verification
Watchlist	routes/watchlist.js	CRUD for saved titles and dashboard statistics
Explore	routes/explore.js	TMDB proxy for trending, search and details; recommendations
Data	models/User.js, models/WatchItem.js	Mongoose schemas

4. Data model

Collection	Fields
users	name, email (unique), password (bcrypt hash), createdAt
watchitems	user (reference to users), tmdbId, title, posterPath, type (movie or tv), status (plan_to_watch, watching, completed, dropped), rating (0 to 10), review, runtime (minutes, from TMDB), addedAt

5. API reference

Method	Path	Auth	Purpose
POST	/api/auth/register	No	Create an account; returns a token
POST	/api/auth/login	No	Log in; returns a token
GET	/api/auth/me	Yes	Current user without the password hash
GET	/api/watchlist	Yes	All saved titles, newest first
GET	/api/watchlist/dashboard	Yes	Statistics, top-rated titles, hero item with backdrop
POST	/api/watchlist	Yes	Add a title; runtime is fetched from TMDB
PUT	/api/watchlist/:id	Yes	Update status, rating or review
DELETE	/api/watchlist/:id	Yes	Remove a title
GET	/api/explore/trending	No*	Trending titles (media_type, time_window, page)
GET	/api/explore/search	No*	Search movies and TV (query, page)
GET	/api/explore/:type/:id/providers	No*	Streaming providers for a title
GET	/api/explore/details/:type/:id	No*	Details with providers and external IDs
GET	/api/explore/smart-recommendations	Yes	Recommendations seeded by the user's best titles

* These routes need TMDB_API_KEY on the server. The routes are not behind the JWT check.

6. Configuration

The server reads *server/.env*. Copy *server/.env.example* and set these values:

Variable	Purpose
PORT	API port (default 5000)
MONGODB_URI	MongoDB connection string
JWT_SECRET	Secret for signing tokens. Required: the server exits if it is missing.
TMDB_API_KEY	TMDB API key. Required for the explore routes.

The client reads *client/.env* with VITE_API_URL, which is optional. Real *.env* files and *node_modules/* are excluded from Git by *.gitignore*.

7. Setup and running

Prerequisites: Node.js 18 or newer, MongoDB (local or Atlas), and a TMDB API key.

```
cd server && npm install
cp .env.example .env # then fill in the values
npm run dev

cd ../client && npm install
npm run dev
```

8. Fixes made in this review

Issue	Fix
Login tokens could fail to verify. The routes signed tokens with a fallback string, and the middleware verified with a different default, so any missing JWT_SECRET broke authentication.	Both now read process.env.JWT_SECRET. The server exits at startup if it is not set.
Search text was pasted into the URL, so characters such as &, # or ? broke the query.	The client now passes the query through Axios params, which encodes it.
The mobile drawer used the deprecated ListItem button prop.	Replaced with ListItemButton.
The TMDB key was in server/.env.example.	Replaced with a placeholder. The old key should be rotated, since it was visible in the file.
No .gitignore, so .env files and node_modules could be committed.	Added .gitignore covering node_modules, .env files and build output.
Inconsistent project names and titles.	Updated package names, the HTML title, the API banner and the README to NextWatch.

9. Known limitations and next steps

The points below are suggestions and have not been changed in the code.

• The recommendation engine uses a random shuffle over TMDB's recommendation lists. It does not use genres or ratings beyond choosing the seeds.

• The streaming alert checks India only (region IN). The region could be a setting.

• There are no automated tests and the client lint script has no ESLint configuration file.

• The poster placeholder points to a third-party image service that may stop working.

• There is no rate limiting on login and registration.

• Tokens are kept in localStorage. Moving them to an HTTP-only cookie would reduce the risk from script injection.

• Deployment is not documented. A Dockerfile and a hosted MongoDB would be the next step.